



Responda às questões sobre as alterações em HTML, DOM, eventos, LocalStorage e JSON.

10 Questões Objetivas com Gabarito

Questão 1: Declaração de Input no HTML

No modelo de código que utiliza referências diretas por ID (ex: `nome.value`), qual é a forma correta de declarar o campo de idade para que ele fique acessível como `idade.value` no script?

☒ **A)** `<input id="idade" type="number" placeholder="Idade">`

☐ **B)** `<input class="idade" placeholder="Idade">`

☐ **C)** `<span id="idade"></span>`

☐ **D)** `<div id="idade"></div>`

✅ **Correto! (Gabarito: A)** O atributo `id="idade"` cria o identificador global no DOM que permite ao JavaScript acessar o elemento diretamente pela variável `idade`, e o `type="number"` restringe para caracteres numéricos.

Questão 2: Inclusão no Cabeçalho da Tabela

No HTML original, temos: `<tr><th>ID</th><th>Nome</th><th>E-mail</th><th>Ações</th></tr>`. Qual alteração mantém a simetria com as linhas de dados?

☒ **A)** Adicionar `<th>Idade</th>` entre E-mail e Ações.

☐ **B)** Adicionar `<th colspan="2">E-mail</th>`.

☐ **C)** Não precisa alterar o cabeçalho, pois a tabela ajusta as colunas automaticamente.

☐ **D)** Substituir E-mail por Idade.

✅ **Correto! (Gabarito: A)** Para que uma tabela mantenha suas colunas devidamente alinhadas, cada célula `<td>` criada no corpo deve possuir seu respectivo `<th>` no cabeçalho na mesma ordem.

Questão 3: Inclusão no Objeto do Aluno (Create)

Na função `salvar()`, na parte que cadastra um novo aluno (`else`), como incluímos a propriedade `idade` no array?

☒ **A)** `alunos.push({ id: proximoId++, nome: nome.value, email: email.value, idade: idade.value });`

☐ **B)** `alunos.push(nome.value, email.value, idade.value);`

☐ **C)** `alunos.idade = idade.value;`

☐ **D)** `alunos.push({ id: proximoId++, dados: [nome, email, idade] });`

✅ **Correto! (Gabarito: A)** Cada aluno é um objeto estruturado em memória. Inserir a chave `idade: idade.value` garante que a propriedade fará parte do registro que será salvo no array.

Questão 4: Limpeza dos Campos após Salvar

O código original esvaziava os campos com `nome.value = email.value = ""`; Para esvaziar também o novo campo de idade, qual linha deve ser usada?

☒ **A)** `nome.value = email.value = idade.value = ""`;

☐ **B)** `idade.reset()`;

☐ **C)** `clear(nome, email, idade)`;

☐ **D)** `idade.value = null`;

✅ **Correto! (Gabarito: A)** O encadeamento de atribuição `nome.value = email.value = idade.value = ""`; redefine o valor de todos os campos para uma string vazia em uma única linha limpa.

Questão 5: Operador Nullish Coalescing na Leitura

Na função `listar()`, utilizamos `${a.idade ?? "-"}` para renderizar a idade. O que o operador `??` faz nesse caso?

☒ **A)** Se a propriedade `idade` for `null` ou `undefined` (ex: registros antigos), exibe `"-"` em vez da palavra `'undefined'`.

☐ **B)** Ele converte automaticamente o texto da idade em número inteiro.

☐ **C)** Ele soma a idade com o próximo ID do aluno.

☐ **D)** Ele lança um erro caso a idade esteja em branco.

✅ **Correto! (Gabarito: A)** O operador `??` (nullish coalescing) fornece um valor de fallback seguro para propriedades inexistentes ou nulas, garantindo compatibilidade com backups antigos.

Questão 6: Carregamento no Modo de Edição

Na função `editar(id)`, qual linha devemos acrescentar para que o valor da idade do aluno selecionado apareça no input para alteração?

☒ **A)** `idade.value = a.idade || "";`

☐ **B)** `a.idade = idade.value;`

☐ **C)** `idade.innerHTML = a.idade;`

☐ **D)** `localStorage.getItem(a.idade);`

✅ **Correto! (Gabarito: A)** Ao clicar em 'Editar', os inputs do formulário devem receber os valores atuais do objeto. A expressão `a.idade || ""` garante que, caso não haja idade gravada, o input fique vazio.

Questão 7: Atualização do Registro Existente (Update)

Quando `idEditando` não for nulo na função `salvar()`, como salvamos a nova idade no objeto encontrado?

☒ **A)** `a.idade = idade.value;`

☐ **B)** `alunos[idEditando] = idade.value;`

☐ **C)** `idade.value = a.idade;`

☐ **D)** `a.push(idade.value);`

✅ **Correto! (Gabarito: A)** A variável `a` armazena a referência do objeto localizado por `alunos.find()`. Atribuir `a.idade = idade.value` atualiza diretamente a propriedade daquele aluno no array.

Questão 8: Comportamento da Função persistir()

A função `persistir()` é declarada como: `function persistir() { localStorage.setItem(CHAVE, JSON.stringify(alunos)); }`. O que precisa ser alterado nela após incluir o campo idade?

- ☒ **A)** Absolutamente nada, pois `JSON.stringify()` serializa automaticamente todas as propriedades de cada objeto do array.
- ☐ **B)** É necessário concatenar `+ idade.value` dentro do `setItem`.
- ☐ **C)** É preciso criar uma segunda chave no LocalStorage chamada 'idades.json'.
- ☐ **D)** Precisa ser reescrita usando um loop `for` manual.

✅ **Correto! (Gabarito: A)** A serialização JSON é universal: qualquer propriedade nova adicionada aos objetos em memória é automaticamente codificada em texto JSON e salva no LocalStorage sem alterar a função.

Questão 9: Exportação e Importação de Arquivos JSON

Ao clicar em '📄 Exportar' e gerar o download do arquivo `alunos.json`, os novos dados de idade estarão presentes?

- ☒ **A)** Sim, porque a exportação pega o array completo `alunos` e o converte em Blob, e a importação com `FileReader` o restaura integralmente.
- ☐ **B)** Não, porque a função `exportar()` precisa de um parâmetro extra para cada coluna nova.
- ☐ **C)** Apenas se o usuário tiver preenchido a idade em todos os alunos.
- ☐ **D)** O navegador rejeitará o arquivo JSON por conter 4 campos.

✅ **Correto! (Gabarito: A)** Como a exportação e importação operam diretamente sobre o array de objetos e texto JSON puro via Blob e `FileReader`, qualquer novo atributo é preservado no arquivo baixado e importado.

Questão 10: Exclusão e Reset

O que acontece com as funções `excluir(id)` e `resetar()` após a inclusão do campo idade?

- ☒ **A)** Continuam funcionando perfeitamente sem nenhuma modificação, pois excluem pelo ID ou limpam a chave inteira do `LocalStorage`.
 - ☐ **B)** Precisam ser atualizadas para deletar a idade separadamente.
 - ☐ **C)** O `filter()` da exclusão falha se o objeto tiver mais de 3 propriedades.
 - ☐ **D)** O `localStorage.removeItem()` não apaga propriedades numéricas.
- ✅ **Correto! (Gabarito: A)** A função `excluir()` filtra o array pelo `id` do aluno (removendo o objeto completo com todas as suas propriedades), e `resetar()` apaga o registro inteiro do `LocalStorage`.

Relatório Oficial de Desempenho e Gabarito

Gerado em: 01/10/2026, 14:12:11

10

ACERTOS

0

ERROS

100%

APROVEITAMENTO

Q1. Declaração de Input no HTML

Sua resposta: **A** | Gabarito: **A (ACERTOU)**

Justificativa: O atributo `id="idade"` cria o identificador global no DOM que permite ao JavaScript acessar o elemento diretamente pela variável `idade`, e o `type="number"` restringe para caracteres numéricos.

Q2. Inclusão no Cabeçalho da Tabela

Sua resposta: **A** | Gabarito: **A (ACERTOU)**

Justificativa: Para que uma tabela mantenha suas colunas devidamente alinhadas, cada célula `<td>` criada no corpo deve possuir seu respectivo `<th>` no cabeçalho na mesma ordem.

Q3. Inclusão no Objeto do Aluno (Create)

Sua resposta: **A** | Gabarito: **A (ACERTOU)**

Justificativa: Cada aluno é um objeto estruturado em memória. Inserir a chave `idade: idade.value` garante que a propriedade fará parte do registro que será salvo no array.

Q4. Limpeza dos Campos após Salvar

Sua resposta: **A** | Gabarito: **A (ACERTO)**

Justificativa: O encadeamento de atribuição `nome.value = email.value = idade.value = ""`; redefine o valor de todos os campos para uma string vazia em uma única linha limpa.

Q5. Operador Nullish Coalescing na Leitura

Sua resposta: **A** | Gabarito: **A (ACERTO)**

Justificativa: O operador `??` (nullish coalescing) fornece um valor de fallback seguro para propriedades inexistentes ou nulas, garantindo compatibilidade com backups antigos.

Q6. Carregamento no Modo de Edição

Sua resposta: **A** | Gabarito: **A (ACERTO)**

Justificativa: Ao clicar em 'Editar', os inputs do formulário devem receber os valores atuais do objeto. A expressão `a.idade || ""` garante que, caso não haja idade gravada, o input fique vazio.

Q7. Atualização do Registro Existente (Update)

Sua resposta: **A** | Gabarito: **A (ACERTO)**

Justificativa: A variável `a` armazena a referência do objeto localizado por `alunos.find()`. Atribuir `a.idade = idade.value` atualiza diretamente a propriedade daquele aluno no array.

Q8. Comportamento da Função `persistir()`

Sua resposta: **A** | Gabarito: **A (ACERTO)**

Justificativa: A serialização JSON é universal: qualquer propriedade nova adicionada aos objetos em memória é automaticamente codificada em texto JSON e salva no `LocalStorage` sem alterar a função.

Q9. Exportação e Importação de Arquivos JSON

Sua resposta: **A** | Gabarito: **A (ACERTO)**

Justificativa: Como a exportação e importação operam diretamente sobre o array de objetos e texto JSON puro via `Blob` e `FileReader`, qualquer novo atributo é preservado no arquivo baixado e importado.

Q10. Exclusão e Reset

Sua resposta: **A** | Gabarito: **A (ACERTO)**

Justificativa: A função `excluir()` filtra o array pelo `id` do aluno (removendo o objeto completo com todas as suas propriedades), e `resetar()` apaga o registro inteiro do `LocalStorage`.